Programming in *Rust*

Lukáš Hozda

NPRG082 · winter 2026/27

Course organization

- Lectures and labs, 90 minutes each
- Six core lecture/lab pairs, plus an extension block
- Bring a laptop to the labs
- Visitors welcome; feel free to share the materials
- *Schedule and course information*

Homework

- ReCodEx; two assignments per core lecture/lab pair
- Executable tasks: `main.rs` and permitted extra files
- Library tasks: `solution.rs`, tested through its API
- Deadlines and grading: course page and assignment details
- Rust 1.88.0 for local testing and graded submissions

Contacts

- *Course Discord*
- *Rust CZ/SK Discord*
- Direct contact with the instructors is also welcome
- Materials and examples: *matfyz.lho.sh*



discord.gg/E4XpeWbJpx

Course topics

- Ownership, borrowing, and data structures
- Enums, pattern matching, and error handling
- Modules, traits, generics, and iterators
- Threads, synchronization, and asynchronous tasks
- Library design, testing, and dependencies throughout

Rust history

- 2006: Graydon Hoare starts the project
- 2009: Mozilla begins sponsoring development
- 2015: Rust 1.0
- 2018: non-lexical lifetimes; 2019: async/await
- 2021: Rust Foundation

Language influences

- C++: resource management and zero-overhead abstractions
- ML-family languages: algebraic data types and matching
- Haskell: traits have similarities to type classes
- Cyclone: memory safety in a systems language
- Earlier Rust also explored Erlang-style concurrency

Design goals

- Memory safety without a mandatory garbage collector
- Control over allocation, layout, and resource lifetime
- Abstractions that can compile to direct machine code
- Data-race prevention in safe code
- Interoperation with existing native libraries

Resource management

- Manual management: explicit allocation and release
- Tracing GC: runtime checks which objects are reachable
- Ownership: a value is responsible for a resource
- Borrowing: access without taking ownership
- Rust combines ownership with static checks on access

Static guarantees

- Reject invalid operations before the program runs
- Check function bodies against their signatures
- Avoid relying on a caller's undocumented behavior
- Some safe programs need a different representation
- Runtime checks where static checks are insufficient

Installing Rust

- Install through *rustup.rs*
- Linux and macOS: a native linker/toolchain is also needed
- Windows: Rustup installer and MSVC build tools
- WSL is an alternative Windows setup
- Use Rustup to manage compiler versions independently

Course toolchain

```
rustup toolchain install 1.88.0
rustup override set 1.88.0
rustc --version
cargo --version
```

- Set the override inside your course directory
- Use the ReCodEx compiler version for local testing
- Nightly features only when an assignment requests them

Editor setup

- VS Code or VSCodium: rust-analyzer
- Neovim or Emacs: rust-analyzer through an LSP client
- RustRover is another option
- CodeLLDB for debugging in VS Code/VSCodium
- TOML support for `Cargo.toml`

Standard tools

```
rustup component add rustfmt clippy rust-src
cargo fmt
cargo clippy
cargo test
cargo doc --open
```

- Formatting, diagnostics, tests, documentation
- `rust-src`: standard-library sources for editor support

Cargo project

```
cargo new demo
cd demo
cargo check
cargo run
cargo run --release
```

- `Cargo.toml`: package metadata and dependencies
- `src/main.rs`: executable entry point
- Debug: faster compilation; release: optimized execution

Crates and packages

- Crate: a library or executable compilation unit
- Package: a `Cargo.toml` and its associated targets
- One package can contain a library and several executables
- Modules organize code within a crate
- Cargo builds the package's dependency graph

Dependencies

```
cargo add regex
```

- `std` covers primitives, not every application domain
- *crates.io*: package registry
- *docs.rs*: published API documentation
- *lib.rs*: an alternative crate directory
- Check maintenance, license, and supported Rust version

First program

```
fn main() {
    println!("Hello, world!");
}
```

- `fn`: function declaration; `main`: entry point
- `println!`: macro with compile-time format checking
- Save as `hello.rs`; compile with `rustc hello.rs`
- Run `./hello` on Unix or `.\hello.exe` on Windows

Values and types

```
fn main() {
    let count: u32 = 42;
    let ratio: f64 = 0.25;
    let ready: bool = true;
    let letter: char = 'λ';
    println!("{count} {ratio} {ready} {letter}");
}
```

- Static types; many local annotations can be inferred
- Bindings are immutable unless declared `mut`

Basic types

- Signed integers: `i8` through `i128`
- Unsigned integers: `u8` through `u128`
- `isize`, `usize`: pointer-sized integers
- `f32`, `f64`: floating-point numbers
- `bool`: one byte; `char`: Unicode scalar value, four bytes
- `()`: unit, a value with no stored data

Data representation

- `[T; N]`: fixed-size array stored inline
- `Vec<T>`: pointer, length, capacity; allocated elements
- `String`: an owned UTF-8 buffer
- `&str`: a borrowed UTF-8 view, not a copied string
- Owning a buffer differs from having a pointer into it

Assignment of an owned value

```
fn main() {
    let text = String::from("hello");
    let other = text;
    println!("{text}");
    println!("{other}");
}
```

Moves

- `text` initially owns the string's allocation
- Assignment transfers ownership to `other`
- The old binding can no longer be used
- The string's contents are not implicitly cloned
- Two independent owners must not free the same allocation

Explicit copying

```
fn main() {
    let text = String::from("hello");
    let other = text.clone();
    println!("{text} / {other}");
}
```

- `clone` copies this string's allocation and contents
- The strings can now be changed independently
- Copying is explicit and has a runtime cost

Borrowed results

```
fn first_word(input: &str) -> &str {
    input.split_whitespace().next().unwrap_or("")
}
fn main() {
    let line = String::from("systems programming");
    println!("{}", first_word(&line));
}
```

- The result may refer into the input's allocation
- Its validity is tied to the input borrow

References and mutation

```
fn main() {
    let mut values = vec![10, 20, 30];
    let first = &values[0];
    values.push(40);
    println!("{first}");
}
```

Vector growth

- `push` may allocate a larger buffer and move the elements
- A reference into the old buffer would become invalid
- `push` requires exclusive access through `&mut self`
- `first` is still used after that operation
- The shared and exclusive access cannot overlap here

Reserved capacity

```
fn main() {
    let mut values = Vec::with_capacity(64);
    values.push(10);
    let first = &values[0];
    values.push(20);
    println!("{first}");
}
```

- Still rejected: the access rules do not track spare capacity

Static checking and API contracts

- Function signatures describe permitted access
- Callers need not inspect the implementation
- Safe implementations can change without breaking callers
- Some valid executions cannot be expressed with this API
- Alternatives: a different representation, a shorter borrow

End of a borrow

```
fn main() {
    let mut values = vec![10, 20, 30];
    let first = &values[0];
    println!("{first}");
    values.push(40);
    println!("{values:?}");
}
```

- No later use of `first`; its borrow can end before `push`

Indices instead of references

```
fn main() {
    let mut values = vec![10, 20, 30];
    let index = 0;
    values.push(40);
    println!("{}", values[index]);
}
```

- Appending preserves earlier indices
- Insertion or removal before an element shifts its index

Resource cleanup

- Ownership applies to more than memory
- `File`: close an owned file handle
- `MutexGuard`: release a lock when the guard is dropped
- Cleanup normally follows scope exit, including early returns
- Leaks are possible; memory safety does not require reclamation

Absence as a value

```
fn main() {
    let values = [10, 20];
    match values.get(5) {
        Some(value) => println!("{value}"),
        None => println!("no element"),
    }
}
```

- `Option<T>` distinguishes a value from its absence
- No invalid reference represents the missing element

Errors as values

```
fn main() {
    let result = "70000".parse::<u16>();
    match result {
        Ok(port) => println!("port {port}"),
        Err(error) => println!("{error}"),
    }
}
```

- `Result<T, E>` includes the error type in the contract
- Callers can handle errors or propagate them with `?`

Error-handling decisions

- Recoverable errors are usually returned as values
- `?` propagates errors through the function's return type
- Ignoring a `Result` produces a warning by default
- `unwrap` can panic; it is an explicit failure policy
- Types do not prove application behavior correct

Data modelling

```
enum Connection {
    Disconnected,
    Connected { peer: String },
}
```

- Different states can contain different data
- A disconnected connection has no peer field to misuse
- Exhaustive matching checks that every case is handled

Exhaustive matching

```
enum State { Stopped, Running }
fn main() {
    let state = State::Running;
    match state {
        State::Running => println!("running"),
    }
}
```

- Rejected: the `Stopped` case is missing
- Adding a variant exposes incomplete matches in callers

Borrowing across a thread boundary

```
fn main() {
    let message = String::from("work");
    let handle = std::thread::spawn(|| message.len());
    println!("{}", handle.join().unwrap());
}
```

- `spawn` permits the new thread to outlive the current scope
- Its closure cannot retain this borrow of a local variable

Ownership transfer to a thread

```
fn main() {
    let message = String::from("work");
    let handle = std::thread::spawn(move || message.len());
    println!("{}", handle.join().unwrap());
}
```

- The closure now owns `message`
- `join` waits for completion and returns the result

Shared state

- Ownership transfer avoids shared mutable state
- Shared ownership is possible, for example with `Arc<T>`
- Mutation may require a mutex or another synchronization scheme
- `Send` and `Sync` describe thread-safety properties
- Data-race freedom does not prevent deadlocks or logic races

Abstraction costs

```
fn main() {
    let total: i32 = (1..=100)
        .map(|n| n * n)
        .sum();
    println!("{total}");
}
```

- No intermediate collection is required by this pipeline
- Iteration and closures can be inlined in optimized builds

Traits and dispatch

- Traits describe required behavior
- Generic code can use static dispatch, with code for each type
- `dyn Trait` permits dynamic dispatch when useful
- Composition without class-inheritance hierarchies
- Runtime costs depend on the abstraction chosen

Runtime costs

- Allocation, cloning, locking, reference counting cost work
- Bounds checks may remain in generated code
- Generic instantiations can increase binary size and compile time
- Debug and release builds can perform very differently
- Zero-overhead abstraction is not a speed guarantee

Unsafe code

- Needed for some low-level operations and native interfaces
- Authors must uphold the operation's safety requirements
- An `unsafe` block does not disable all compiler checks
- Safe APIs can encapsulate an unsafe implementation
- An implementation bug can invalidate its safety claim

Compiler diagnostics

- Primary error: the operation the compiler rejected
- Source annotations: related uses and conflicting requirements
- Error explanations: `rustc --explain E0502`
- Suggestions are possible changes, not always the right design
- Short examples separate type rules from application details

Before the next lab

- Install Rust 1.88.0 and configure your editor
- Build and run a Cargo project
- Bring a laptop
- Course Discord and materials are linked at the start

References

- *The Rust Programming Language*
- *Rust by Example*
- *Standard library documentation*
- *The Cargo Book*
- *The Rustup Book*